1. The Big Picture

A simple Data Engineering flow can be understood as:

Source File
↓
Spark reads it
↓
DataFrame
↓
Transform / Validate
↓
Persist Data
↓
Parquet / Delta
↓
Storage Location

The important idea is that **processing data and storing data are different things**.

Spark processes the data, but Spark itself is not the permanent storage system.

---

2. Where Can Source Data Come From?

A Data Engineering pipeline may receive data from:

- CSV files
- JSON files
- Parquet files
- Databases
- APIs
- Cloud storage
- Streaming systems

Regardless of the source, the data ultimately needs to exist somewhere in storage before or after processing.

---

3. File vs DataFrame

This distinction is very important.

File

A file is data physically stored somewhere.

Examples:

orders.csv
customers.json
transactions.parquet

The file continues to exist even after Spark code finishes.

DataFrame

A DataFrame is Spark's structured representation of data while processing it.

It contains rows and columns and can be used for:

- Filtering
- Selecting columns
- Joining
- Aggregating
- Transforming

Remember

FILE → Stored data
DATAFRAME → Spark's working representation of that data

Creating a DataFrame does **not** automatically create a table or permanently store transformed data.

---

4. Reading a CSV with Spark

Suppose the source system provides an orders CSV file.

```
orders_df = (
   spark.read
   .option("header", True)
```

```
    .option("inferSchema", True)
    .csv(orders_path)
)
```

What Each Part Means

spark.read

Spark starts reading data.

.option("header", True)

The first CSV row contains column names.

.option("inferSchema", True)

Spark tries to identify the data types.

.csv(orders_path)

The source format is CSV.

View the data:

display(orders_df)

Check the schema:

orders_df.printSchema()

After reading the file, Spark can now work with it as a DataFrame.

---

5. Example Orders Dataset

A typical orders dataset may contain:

order_id
customer_id
product_id
order_date
quantity
unit_price
order_status

For example:

- `order_id` → unique order
- `customer_id` → customer identifier
- `product_id` → purchased product
- `order_date` → order date
- `quantity` → number of units
- `unit_price` → price per unit
- `order_status` → `COMPLETED`, `PENDING`, or `CANCELLED`

In a real project, such a file could arrive from another application and land in cloud storage before being processed.

---

6. Why CSV Is Not Ideal for Everything

CSV is useful because it is:

- Simple
- Easy to understand
- Easy to exchange between systems

But it has limitations.

- It is text-based.
- It does not strongly preserve schema by itself.
- It can require more parsing during analytics.
- It can use more storage than efficient columnar formats in many workloads.
- It does not provide advanced transactional table capabilities.

So CSV is useful for exchanging data, but analytical pipelines often use better storage formats after ingestion.

---

7. What Is Parquet?

Parquet is a **columnar file format** commonly used for analytical workloads.

Important characteristics:

- Column-oriented storage
- Preserves schema information
- Supports efficient compression
- Works well with Spark

Suppose a dataset contains 50 columns but your query needs only:

order_date
unit_price

Column-oriented storage is designed to make this kind of analytical access more efficient.

---

8. Writing a DataFrame as Parquet

```
orders_df.write \
    .mode("overwrite") \
    .parquet(parquet_path)
```

This means:

DataFrame
  ↓
Persist
  ↓
Parquet Files

Important:

Writing Parquet does **not** automatically create a table.

Spark writes the data to the specified storage location.

---

9. Why Spark May Create Multiple Files

Students may expect:

orders.parquet

But Spark may write multiple part files.

Why?

Because Spark is a distributed processing engine.

Data can be processed and written from multiple partitions.

So the output of a Spark write commonly contains multiple files rather than one single file.

---

10. Reading Parquet Back

parquet_df = spark.read.parquet(parquet_path)

display(parquet_df)

Check schema:

parquet_df.printSchema()

The important flow is:

CSV
↓
DataFrame
↓
Processing
↓
Parquet
↓
Persisted Data

Parquet data can be read again later, and schema information is available with it.

---

11. What Is Delta?

Delta Lake is a storage layer commonly used with Databricks.

At this stage, remember:

Delta = Parquet Data Files + Transaction Metadata

That additional metadata enables capabilities beyond plain Parquet.

The deeper concepts such as transaction logs, Time Travel, RESTORE, and MERGE are covered separately.

---

12. Writing Data as Delta

```
orders_df.write \
    .format("delta") \
    .mode("overwrite") \
    .save(delta_path)
```

Meaning

orders_df.write

DataFrame to persist.

.format("delta")

Use Delta format.

.mode("overwrite")

Replace existing data at the demo location.

.save(delta_path)

Write the data to the storage location.

---

13. `_delta_log`

After writing Delta data, inspecting the location may show:

_delta_log

This is associated with how Delta tracks changes and transactions.

For this class, the key point is simply:

Delta contains data files plus Delta-specific metadata.

---

14. Reading Delta Data

```
delta_df = (
    spark.read
    .format("delta")
    .load(delta_path)
)

display(delta_df)
```

DataFrame vs Delta

DataFrame
→ Spark representation used during processing

Delta
→ Persisted data stored on storage
→ Can be read again later
→ Includes Delta-specific metadata

A useful way to remember it:

> A DataFrame is how Spark works with data.
> Delta is one way the processed data can be persisted.

---

15. CSV vs Parquet vs Delta

Concept	CSV	Parquet	Delta
Type	File format	Columnar file format	Storage layer/format used with Databricks
Text-based	Yes	No	Uses Parquet data files
Schema information	Limited	Preserved	Preserved

Analytical workloads	Basic	Efficient	Designed for Data Engineering workloads
Extra metadata	No	Basic schema metadata	Delta transaction metadata

The goal at this stage is to understand the progression:

CSV
↓
DataFrame
↓
Parquet / Delta

---

16. Why Paths Alone Become Difficult

Reading Delta using:

```
spark.read
   .format("delta")
   .load(delta_path)
```

works.

But imagine having hundreds or thousands of datasets.

You would need to remember:

- Where orders data is stored
- Where customer data is stored
- What format each dataset uses
- What columns it contains
- What schema it has
- Which path represents which business dataset

This quickly becomes difficult.

Instead of remembering:

some/storage/path/project/ecommerce/orders_delta/

we would prefer:

SELECT * FROM orders;

But then Databricks must know what `orders` means.

That requires **metadata**.

---

17. What Is Metadata?

A simple definition:

> **Metadata is data about our data.**

For an orders dataset, metadata could include:

Name → orders
Format → Delta
Columns → order_id, customer_id, product_id...
Location → storage path

Data vs Metadata

Actual order records:

DATA

Information describing those records:

Table name
Columns
Schema
Location
Format

is:

METADATA

Metadata makes datasets easier to identify and manage.

---

18. Why Tables Are Needed

Suppose your data is physically stored at:

/project/ecommerce/orders_delta/

Without a table, you may need to remember that path.

With proper metadata, you want to work with something like:

SELECT * FROM orders;

So tables give a meaningful name to data, while metadata helps the system understand:

- Where the data exists
- What format it uses
- What columns it contains
- What schema it follows

This is why file-based storage eventually leads to the concept of tables and metastores.

---

19. Important Distinctions

Remember these four concepts:

FILE

Physical data stored somewhere.

DATAFRAME

Spark representation used to process data.

PARQUET / DELTA

Formats used to persist data.

METADATA

Information describing the data.

This distinction is more important than memorizing commands.

---

20. Common Errors

Path Not Found

Possible error:

PATH_NOT_FOUND

Check:

- File path
- File existence
- Path spelling

Reading CSV as Delta

Wrong:

spark.read.format("delta").load(orders_path)

if the source is a CSV.

Correct:

spark.read.csv(orders_path)

The reader format must match the actual source format.

Existing Output Path

During learning:

.mode("overwrite")

can replace existing output.

But do not blindly use `overwrite` in production systems.

---

21. End-to-End Flow to Remember

orders.csv
↓
Spark reads the file
↓
orders_df
↓
Transform / Validate
↓
Persist
↓
Parquet / Delta
↓
Storage Location
↓
Need a meaningful name + metadata
↓
Tables / Metastore

This is the core flow of the class.

---

22. Student Practice

Create:

customers.csv

with:

customer_id,customer_name,city
C101,Anurag,Bengaluru
C102,Rahul,Pune
C103,Priya,Delhi
C104,Neha,Mumbai
C105,Amit,Hyderabad

Complete these tasks:

1. Read the CSV as a DataFrame.
2. Display the DataFrame.
3. Print its schema.
4. Write the DataFrame as Parquet.
5. Read the Parquet data again.
6. Write the DataFrame as Delta.
7. List the Delta storage location.
8. Identify `_delta_log`.

Do not create a table yet.

---

23. Interview-Relevant Questions

1. What is the difference between a file and a DataFrame?

A file is physically stored data. A DataFrame is Spark's structured representation of data used during processing.

---

2. Does creating a DataFrame persist the data?

No.

Creating a DataFrame does not automatically store transformed data permanently.

---

3. Why is Parquet preferred over CSV for many analytical workloads?

Parquet is columnar, preserves schema information, supports efficient compression, and works well with analytical processing engines such as Spark.

---

4. Why can Spark write multiple Parquet files?

Spark is distributed and can write output from multiple partitions, which commonly results in multiple part files.

---

5. What is Delta Lake at a beginner level?

Delta is a storage layer commonly used with Databricks that uses Parquet data files together with transaction metadata.

---

6. What is `_delta_log`?

It is Delta-specific metadata related to tracking changes and transactions.

---

7. What is the difference between a DataFrame and Delta data?

A DataFrame is used by Spark while processing data.

Delta represents persisted data stored on storage with Delta-specific metadata capabilities.

---

8. What is metadata?

Metadata is information that describes data, such as:

- Dataset/table name
- Columns
- Schema
- Format
- Storage location

---

9. Why are tables needed if data already exists in storage?

Managing hundreds of datasets only by storage paths becomes difficult.

Tables provide meaningful names and metadata so users can work with datasets more easily.

---

10. Does writing Parquet or Delta automatically create a table?

No.

Writing data to a path persists the files, but a table is a separate logical/metadata concept.

---

24. Quick Revision

File
↓
Physical stored data

DataFrame
↓
Spark's processing representation

Parquet / Delta
↓
Persisted storage formats

Metadata
↓
Description of the stored data

Table
↓
Meaningful logical representation backed by metadata